

Bonnes Pratiques en Calcul Scientifique

Contexte et problématique

Les scientifiques consacrent plus de **30 %** de leur temps à développer des logiciels, mais plus de **90 %** sont autodidactes. Ils manquent ainsi des pratiques fondamentales : code maintenable, contrôle de version, tests unitaires, automatisation.

Les auteurs soutiennent que le logiciel scientifique est un **instrument expérimental** et doit être construit, vérifié et utilisé avec la même rigueur qu'un équipement de laboratoire.

Les 8 grandes pratiques recommandées

1. Écrire pour les humains, pas pour la machine

- Limiter la charge mémorielle du lecteur : décomposer les programmes en fonctions courtes à tâche unique.
- Nommer les variables de façon cohérente, distinctive et significative (éviter "a", "foo", "results2").
- Adopter un style de code uniforme (indentation, capitalisation) dans tout le projet.

2. Laisser l'ordinateur faire le travail

- Automatiser les tâches répétitives plutôt que de les répéter manuellement.
- Sauvegarder les commandes dans des scripts réutilisables.
- Utiliser un outil de build (ex : Make) pour automatiser les workflows et garantir que les sorties sont toujours à jour.

3. Travailler par changements incrémentaux

- Avancer par petites étapes avec retour fréquent plutôt que de planifier sur des mois.
- Utiliser un système de contrôle de version (Git, Mercurial, SVN) : il archive l'historique, facilite la collaboration et permet de revenir en arrière.
- Versionner tout ce qui est créé manuellement : code, données brutes, sources des articles.

4. Ne pas se répéter (principe DRY)

- Chaque donnée doit avoir une seule représentation canonique (éviter les constantes dupliquées).
- Modulariser le code au lieu de copier-coller : corriger un bug corrige tous les endroits à la fois.
- Réutiliser des bibliothèques existantes plutôt que de réinventer la roue (numpy, scipy, etc.).

5. Anticiper les erreurs

- Ajouter des assertions pour vérifier les préconditions et postconditions des fonctions.
- Utiliser une bibliothèque de tests unitaires (pytest, unittest) : tests unitaires, d'intégration, de régression.
- Transformer chaque bug découvert en cas de test pour éviter qu'il ne réapparaisse.
- Utiliser un débogueur symbolique pour inspecter l'état du programme pas à pas.

6. Optimiser seulement après que le code fonctionne

- D'abord faire fonctionner correctement, puis identifier les goulots d'étranglement avec un profileur.
- Coder dans le langage de haut niveau le plus adapté ; descendre en C/Fortran seulement si nécessaire.

7. Documenter le pourquoi, pas le comment

- Documenter les interfaces et les raisons des choix, pas les détails d'implémentation.
- Refactoriser le code complexe plutôt que d'écrire de longues explications.
- Intégrer la documentation dans le code source (docstrings, Sphinx, Javadoc).

8. Collaborer

- Pratiquer la revue de code avant fusion (pre-merge code review) : méthode la plus rentable pour trouver des bugs.
- Utiliser la programmation en binôme pour former de nouveaux membres ou résoudre des problèmes difficiles.
- Utiliser un outil de suivi des issues (GitHub Issues, Trac) pour organiser les tâches.

Conclusion

Ces 24 recommandations ne garantissent pas un développement sans erreur, mais utilisées ensemble elles **réduisent les bugs**, facilitent la réutilisation et font gagner du temps. Elles sont aussi les prérequis de la recherche computationnelle reproductible.